

La escalera de los signos

El teorema de conteo de raíces de Sturm, verificado por máquina en Lean 4

Carles Marín^{*}

Investigador independiente

karlesmarin@gmail.com

Junio 2026

Resumen

Presento una formalización completa, libre de `sorry`, en Lean 4 sobre Mathlib, del *teorema de Sturm* (1829): para un polinomio real sin factores cuadrados (*squarefree*) p y un intervalo $(a, b]$ cuyos extremos no son raíces, el número de raíces reales distintas de p en $(a, b]$ es igual a

$$V(a) - V(b),$$

donde $V(x)$ es el número de cambios de signo en la *sucesión de Sturm* $p, p', -(p \bmod p'), \dots$ evaluada en x (descartando los ceros). No se localiza ninguna raíz; se restan dos enteros. La matemática es enteramente clásica y el resultado ya se había formalizado en otros sistemas (Coq, Isabelle/HOL, HOL Light); hasta donde sé, esta es la primera prueba del teorema de Sturm en Lean. La contribución es, por tanto, la formalización misma y la maquinaria reutilizable que requirió: una pequeña teoría de variación de signo, una relación inductiva de “reducción por flancos” que desacopla la combinatoria de la cadena de su álgebra, y el paso de lo local a lo global, de un único cruce de raíz al conteo en el intervalo. El teorema central `Sturm.sturm` depende solo de los tres axiomas estándar `propext`, `Classical.choice`, `Quot.sound`. Un subproducto: el conteo de variaciones de signo de los coeficientes que Mathlib ya tenía (la regla de Descartes) y el que se usa aquí son, tras desplegar las definiciones, *la misma función*—de modo que el instrumental se transfiere literalmente a Descartes.

Qué encontrará aquí. Es un recorrido breve y guiado, no un manual de referencia. Se organiza en torno a cuatro preguntas, cada una abre una sección y entrega al lector la siguiente. La primera pregunta cómo es posible que dos enteros cuenten raíces (Sección 2); la segunda, por qué el conteo solo se mueve en las raíces, y por exactamente uno (Sección 3); la tercera, dónde se resiste de verdad la prueba a volverse formal, y qué enseña esa resistencia (Sección 4); la cuarta, qué queda gratis una vez el teorema está en la biblioteca (Sección 5). Los lemas que sostienen todo están reunidos en una tabla (Sección 7, Tabla 2); los límites honestos—la hipótesis *squarefree*, el cuerpo base, lo que *no* se probó, y las formalizaciones previas en otros sistemas—se exponen sin barniz en la Sección 8; y la sección de cierre pregunta, a modo de reflexión, qué le falta todavía a este artículo. Si solo lee dos cosas, lea el Teorema 1 y la Figura 2; lo demás es el argumento que los conecta, y el argumento es lo importante.

^{*}Formalizado con la asistencia de un sistema de IA (Claude, Anthropic); la matemática es de Sturm, y toda afirmación, alcance y limitación son responsabilidad del autor. La frontera de la contribución se expone con claridad en la Sección 8, y es el núcleo (*kernel*) de Lean—no la asistente—quien certifica las pruebas.

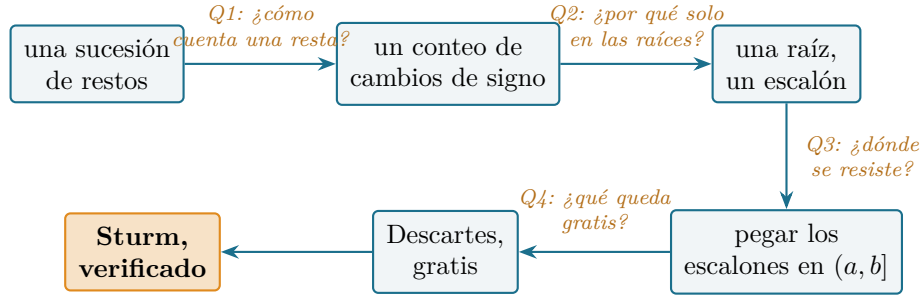


Figura 1: El mapa del lector. De una sucesión de restos a un conteo certificado de raíces y a lo que produce después; cada flecha es la pregunta que responde su sección. Cada nodo es un bloque de Lean libre de `sorry`.

1. La pregunta que lo inicia

¿Cuántas raíces reales tiene un polinomio entre aquí y allá—y puede decirse sin encontrar ni una sola de ellas?

La pregunta es antigua y fue, durante mucho tiempo, genuinamente difícil. La regla de los signos de Descartes (1637) [1] lee una *cota* a partir de los signos de los coeficientes; Budan y Fourier (principios del siglo XIX) [2] la afinaron a una cota para un intervalo; pero una cota no es un conteo, y la distancia entre ambos resistió a los mejores analistas de la época. La cerró en 1829 un joven Charles-François Sturm, que anunció—y en 1835 publicó íntegramente [3]—un procedimiento que devuelve el número *exacto* de raíces reales distintas en cualquier intervalo, sin aproximación alguna y sin localizar raíces. El método fue celebrado de inmediato; es el antepasado del aislamiento de raíces reales, de la descomposición cilíndrica algebraica y de los procedimientos de decisión para la teoría de primer orden de los reales que crecieron a partir del trabajo de Tarski. Tres siglos de esfuerzo por contar raíces terminan, en efecto, en una resta.

El mecanismo es un pequeño giro sobre algo aún más viejo. El algoritmo de Euclides, aplicado a p y su derivada p' , produce una sucesión de restos; el giro de Sturm es negar cada resto al formarlo,

$$p_0 = p, \quad p_1 = p', \quad p_{k+1} = -(p_{k-1} \bmod p_k),$$

deteniéndose cuando el resto se anula. Llámese a esto la *sucesión de Sturm*. Para un punto x que no es raíz de ningún miembro, sea $V(x)$ el número de veces que cambia el signo al leer $p_0(x), p_1(x), \dots$ de izquierda a derecha, ignorando los ceros. El teorema de Sturm afirma que V es, disfrazado, un contador de raíces:

$$\#\{\text{raíces reales distintas de } p \text{ en } (a, b]\} = V(a) - V(b).$$

El resultado es de manual, y se ha verificado por máquina antes: en Coq, como parte de la construcción de los números algebraicos reales de Cohen [5]; en Isabelle/HOL, por Eberl [6] y—en la forma más fina de Sturm–Tarski, vía el índice de Cauchy—por Li y Paulson [7, 8]; y en HOL Light. Lo que no se había hecho, hasta ahora, es en *Lean*. Mathlib, la biblioteca sobre la que se construye buena parte de la matemática formalizada contemporánea, lleva la regla de los signos de Descartes (la función `Polynomial.signVariations`) pero no la sucesión de Sturm ni el teorema de Sturm. Este artículo cubre esa carencia. Quiero ser exacto: es un primero-en-Lean y no un primero-en-cualquier sistema; el valor está en añadir una herramienta fundamental a una biblioteca que no la tenía, y en las piezas reutilizables que la prueba deja tras de sí.

No llegué a ello proponéndome demostrar el teorema de Sturm, sino preguntando, una y otra vez, qué resultados clásicos le faltan aún a una biblioteca madura—y el de Sturm, situado de lleno en el conteo de raíces reales, estaba entre las ausencias más llamativas. El resto del artículo es el camino desde la ausencia hasta el teorema, contado como las cuatro preguntas que un lector cuidadoso probablemente se haga, una tras otra.

2. Q1: ¿cómo puede una resta de enteros contar raíces?

Apunte histórico. La sucesión y el enunciado del conteo son del propio Sturm ([3]; el tratamiento moderno es el de Basu–Pollack–Roy [4]). Nada en esta sección es matemática nueva; el Lean está en `Sturm.lean`, definiciones `sturmSeq` y `signVarAt`.

Precisemos los dos ingredientes. La sucesión de Sturm se construye con la recursión euclidiana negada de arriba; en Lean es `sturmSeq p`, una `List` de polinomios producida por una auxiliar `sturmAux` que recurre sobre el grado del divisor y termina porque cada resto tiene grado estrictamente menor. El conteo de signos es

$$V(x) = \text{signVarAt } (\text{sturmSeq } p) \ x,$$

definido como el número de cambios de signo en la lista de signos evaluados $(\text{sign } p_0(x), \text{sign } p_1(x), \dots)$ tras borrar los ceros. (En la fuente Lean ese borrar-y-contar se expresa con `List.destutter`; una forma recursiva equivalente y más cómoda, `signChanges`, se prueba igual a aquella y es la que usa el resto del desarrollo.)

Teorema 1 (teorema de Sturm, en Lean 4; `Sturm.sturm`). *Sea p un polinomio real squarefree y $a < b$ con $p(a) \neq 0$ y $p(b) \neq 0$. Entonces*

$$V(a) - V(b) = \#\{x : a < x \leq b, p(x) = 0\},$$

el número de raíces reales distintas de p en el intervalo semiabierto $(a, b]$. El enunciado es libre de `sorry`; `#print axioms Sturm.sturm` reporta solo `propext`, `Classical.choice`, `Quot.sound`.

Que una diferencia de dos enteros sea igual a un conteo es, a primera vista, un pequeño milagro, así que conviene verlo ocurrir una vez a mano.

Un caso comprobable a mano. Tomemos $p(x) = x^3 - x = x(x-1)(x+1)$, con raíces $-1, 0, 1$. La recursión euclidiana negada da la sucesión de cuatro términos

$$p_0 = x^3 - x, \quad p_1 = 3x^2 - 1, \quad p_2 = \frac{2}{3}x, \quad p_3 = 1.$$

Leámos los signos en los dos extremos de $(-2, 2]$:

x	p_0	p_1	p_2	p_3	V
-2	$-$	$+$	$-$	$+$	3
2	$+$	$+$	$+$	$+$	0

de modo que $V(-2) - V(2) = 3 - 0 = 3$, exactamente el número de raíces en $(-2, 2]$. Nótese que no se calculó ninguna raíz; solo se leyeron signos. (Cada entrada de esta tabla se comprueba en aritmética exacta con el guion que acompaña a la Figura 2.)

La imagen tras la tabla es la que da nombre a este artículo. Dibújese $V(x)$ a medida que x barre la recta: es una *escalera*, plana casi en todas partes y que baja en uno cuando x pasa por cada raíz de p (Figura 2; la Tabla 1 la lee punto a punto). El teorema de Sturm no es entonces sino leer el descenso total de la escalera entre a y b . Toda la dificultad—y todo el contenido de las dos secciones siguientes—está en probar que la escalera tiene exactamente esta forma: que es plana salvo en las raíces, y que cada escalón mide exactamente uno.

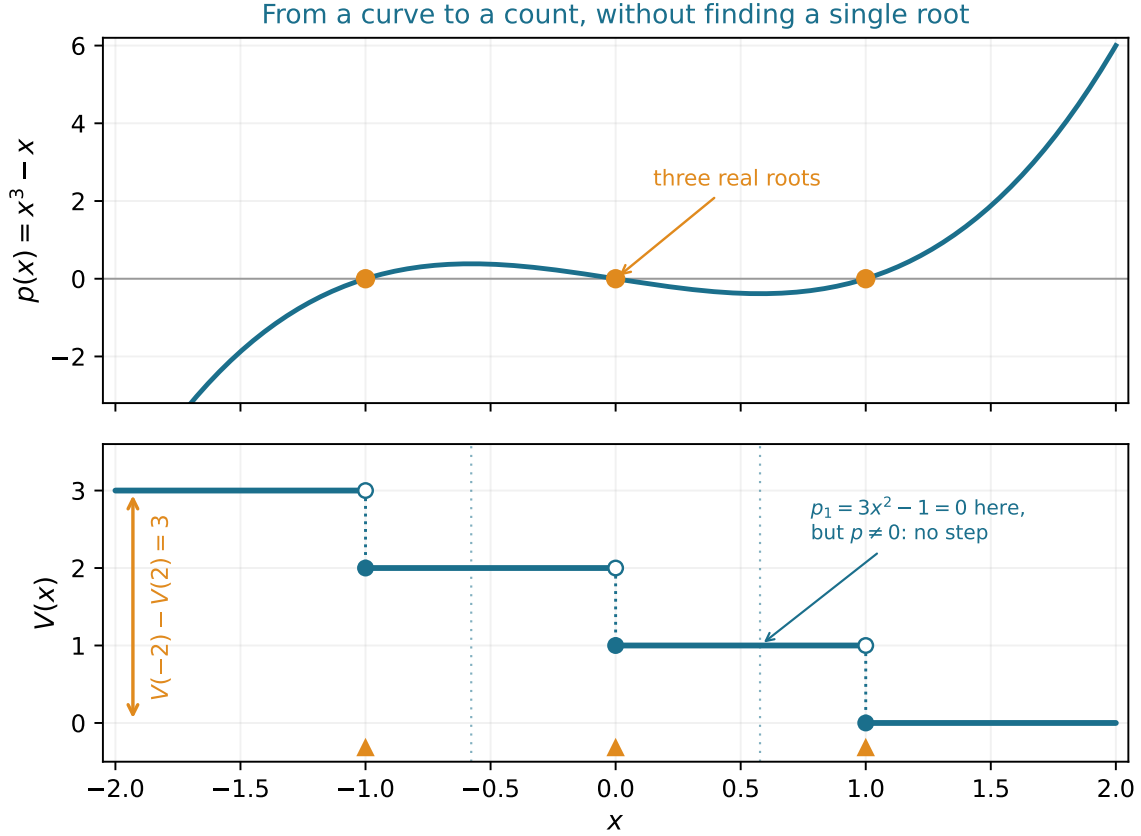


Figura 2: La escalera de los signos para $p = x^3 - x$. *Arriba*: el polinomio y sus tres raíces reales. *Abajo*: el conteo $V(x)$ es plano salvo en las raíces $-1, 0, 1$, donde baja en exactamente uno (círculo abierto = el valor justo a la izquierda, círculo lleno = el valor en la raíz, en el escalón inferior). En un cero de un miembro *interior* de la sucesión—aquí $p_1 = 3x^2 - 1$ se anula en $x = \pm 1/\sqrt{3}$ (guías punteadas), puntos donde p no es cero—la escalera *no* se mueve. El descenso total sobre $(-2, 2]$ (la flecha ámbar) es el número de raíces allí. El guion generador comprueba cada signo y conteo dibujado contra aritmética racional exacta antes de dibujar. (Las etiquetas de la figura están en inglés, compartidas con la edición inglesa.)

x	$p_0 = x^3 - x$	$p_1 = 3x^2 - 1$	$p_2 = \frac{2}{3}x$	$p_3 = 1$	$V(x)$
-2	-	+	-	+	3
-1 (raíz)	0	+	-	+	2
$-\frac{1}{2}$	-	-	-	+	2
$-\frac{1}{\sqrt{3}}$ ($p_1=0$)	-	0	-	+	2
0 (raíz)	0	-	0	+	1
$\frac{1}{\sqrt{3}}$ ($p_1=0$)	+	0	+	+	1
1 (raíz)	0	+	+	+	0
2	+	+	+	+	0

Cuadro 1: El mismo ejemplo leído en los signos. Cada fila evalúa los cuatro miembros de la cadena y cuenta los cambios de signo (ceros omitidos). V baja en uno en cada *raíz* de p y no cambia al cruzar cada cero del miembro *interior* p_1 (las filas $\frac{1}{\sqrt{3}}$)—los dos comportamientos que explica la Sección 3. Toda la tabla se regenera y comprueba en aritmética exacta con el guion de la figura.

3. Q2: ¿por qué el conteo solo se mueve en las raíces, y por exactamente uno?

Apunte histórico. El análisis local—qué le pasa a V cuando x cruza un punto—es el corazón de toda prueba del teorema de Sturm desde la de Sturm. Los dos hechos en que se apoya (un miembro y sus vecinos no pueden anularse juntos; miembros consecutivos son antipodales en un cero del que está entre ellos) son clásicos; el Lean está en los lemas `not_common_root`, `sign_neighbours_opposite_at_interior_root` y `signVarAt_drop_at_critical_point`.

Llámese *crítico* a un punto donde se anula algún miembro de la sucesión. Lejos de los puntos críticos cada miembro conserva su signo (un polinomio no puede cambiar de signo sin una raíz, por el teorema del valor intermedio), así que V es localmente constante: este es `signVarAt_eq_of_no_root`, y es por lo que la escalera es plana entre críticos. Toda la acción está en los puntos críticos, y los hay de dos clases.

Las raíces de p . Supóngase $p(c) = 0$. Como p es squarefree no comparte raíz con p' , luego $p'(c) \neq 0$, y justo a izquierda y derecha de c el polinomio p toma signos opuestos (cruza el cero transversalmente, con el signo de $p'(c)$ a la derecha y el opuesto a la izquierda). El primer par de la sucesión, (p, p') , cambia por tanto en exactamente uno su número de acuerdos internos de signo al cruzar c . Este es el único escalón de la escalera, aislado en Lean como la caída del par de cabeza `signVarAt_cons_head_drop`, alimentado por los lemas de transversalidad `sign_eval_eq_sign_deriv_right` y `sign_eval_eq_neg_sign_deriv_left`.

Los demás puntos críticos. Un miembro p_k con $1 \leq k$ también puede anularse en c sin que p lo haga. Aquí está el mecanismo que hace funcionar el teorema de Sturm, y merece enunciarse como el lema indispensable:

Lema 1 (vecinos antipodales; `sign_neighbours_opposite_at_interior_root`). *Si miembros consecutivos p_{k-1}, p_k son coprimos y $p_k(c) = 0$, entonces $p_{k-1}(c)$ y $p_{k+1}(c)$ son ambos no nulos y de signo opuesto.*

La razón es una consecuencia de una línea de la recursión definitoria $p_{k+1} = -(p_{k-1} \bmod p_k)$ evaluada en una raíz de p_k : allí $p_{k-1}(c) = -p_{k+1}(c)$ (lema `eval_eq_neg_next_of_root`), y ninguno puede ser cero porque polinomios coprimos no comparten raíz (`not_common_root`). En consecuencia, sea cual sea el signo que adopte p_k a cada lado de c —y sea positivo, negativo, o exactamente cero en c —queda entre dos vecinos fijos y opuestos, y un valor encajado entre signos opuestos no aporta nada al conteo de cambios de signo, sea cual sea. El miembro de en medio es invisible. Por eso un cero de un miembro interior no mueve la escalera en absoluto (las marcas $p_1 = 0$ de la Figura 2); la coprimalidad de miembros consecutivos, que se propaga por la cadena desde $\gcd(p, p') = 1$, lo garantiza en todas partes.

Juntando las dos clases se obtiene el cuanto por punto—la altura de un único escalón:

Proposición 1 (un punto crítico; `signVarAt_drop_at_critical_point`). *Si c es el único punto crítico en $[a, b]$ y $p(a), p(b) \neq 0$, entonces*

$$V(a) = V(b) + \begin{cases} 1 & p(c) = 0, \\ 0 & p(c) \neq 0. \end{cases}$$

La prueba es donde la formalización se gana el sueldo, y es el tema de la sección siguiente, porque hacerla rigurosa exigió un recurso que el manual no necesita.

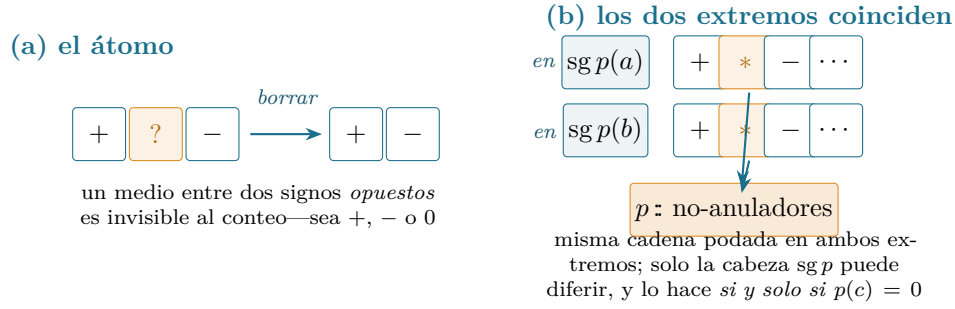


Figura 3: El desacoplo de la Sección 4. (a) El átomo combinatorio: una entrada flanqueada por signos no nulos opuestos (*, el anulador interior) puede borrarse sin cambiar el número de cambios de signo, sea cual sea su propio valor. (b) Aplicado a lo largo de la cadena en ambos extremos, todo anulador interior en c se borra, de modo que las listas de signos en a y en b se reducen a la misma cadena podada; la única entrada que puede diferir es la cabeza $sg\ p$. El álgebra de la cadena suministra las hipótesis de “vecinos opuestos”; el conteo lo mueve **FlankReduce**, que nunca menciona un polinomio.

4. Q3: ¿dónde se resiste la prueba a volverse formal?

Apunte histórico. Sobre el papel, “los miembros interiores se cancelan y solo importa p ” es una frase. Hecha formal, es el único paso genuinamente estructural, porque la cancelación ha de ejecutarse *a lo largo de una lista cuya forma es ella misma recursiva*. El recurso que lo resuelve (**FlankReduce** y **flankReduce_chain_walk**) es, hasta donde sé, la única parte de este desarrollo sin un antecedente directo de manual—no un teorema nuevo, sino una pieza de ingeniería de pruebas.

La dificultad es precisa. Para comparar $V(a)$ con $V(b)$ a través de un único punto crítico c , uno quiere borrar de la lista de signos, en *ambos* a y b , cada miembro interior que se anula en c —cada uno encajado entre vecinos opuestos, de modo que borrarlo no cambia nada (Lema 1)—y observar entonces que las dos listas podadas difieren solo en la cabeza p . Pero “borrar cada tal miembro” es una inducción sobre la sucesión de Sturm, y las hipótesis que justifican un borrado (los vecinos flanqueantes son no nulos y opuestos *en el punto de evaluación*) van entrettejidas en la propia estructura de la cadena. La combinatoria de podar una lista y el álgebra de por qué cada poda es lícita están enredadas, y una prueba honesta debe separarlas.

La separación es una relación inductiva. Diré que una lista de signos *se reduce por flancos* a otra cuando la segunda se obtiene de la primera borrando repetidamente una entrada situada entre dos signos no nulos opuestos:

El desacoplo (receta)

Defínase **FlankReduce** como la relación reflexiva generada por una regla: de $pre ++ [a, b] ++ rest$ se puede pasar a $pre ++ [a, m, b] ++ rest$ siempre que $a, b \neq 0$ y $a \neq b$. Dos hechos reparten entonces el trabajo con limpieza. *Combinatorio*: la reducción por flancos preserva el número de cambios de signo (**FlankReduce.signChanges_eq**)—esto es pura álgebra de listas, sin polinomios. *Estructural*: la lista de signos evaluados de la cadena de Sturm en cualquier punto se reduce por flancos a la cadena sin sus anuladores interiores en c (**flankReduce_chain_walk**)—esta es la inducción sobre la cadena, que suministra las hipótesis de flanco desde la coprimidad y la constancia local del signo, y nada más.

Separadas las dos mitades, la Proposición 1 se vuelve ensamblaje: la lista de signos de la cadena se reduce por flancos, en a y en b por igual, a la *misma* cadena podada p : (no-anuladores); en esa cadena podada solo p puede cambiar de signo entre a y b , de modo que el cambio en V es la única caída de cabeza cuando $p(c) = 0$ y nada en otro caso. La estructura de la cadena entra solo para entregar al lema combinatorio sus hipótesis; el conteo lo mueve una relación que nada sabe de polinomios.

Lo que la máquina me hizo enunciar bien. Hay aquí una lección más callada, del tipo que un asistente de pruebas extrae bien. La prueba clásica supone tácitamente “posición genérica”: que ningún miembro de la sucesión se anula en los extremos a, b , y que siempre se puede deslizar a tales puntos. Formalizar prohíbe el gesto a mano—y resulta que la reducción por flancos ya trata los casos incómodos gratis. Un miembro que se anula justo en un punto de evaluación sigue encajado allí entre vecinos opuestos, así que sigue borrándose por la misma relación; la cadena podada es la misma; el conteo no se ve afectado. Por eso probé el lema por punto (Proposición 1) para un punto crítico en cualquier lugar del intervalo cerrado $[a, b]$, extremos incluidos, y no solo estrictamente dentro—y los casos degenerados que el manual pasa por alto se despachan con la misma línea que el caso principal. El principio transferible es el mismo que extrajo el artículo compañero sobre las desigualdades de Newton [11]: enunciar un paso al nivel más débil que sea honesto a menudo disuelve, en vez de multiplicar, los casos límite.

De un escalón a toda la escalera. El enunciado global es entonces una inducción sobre el número de puntos críticos en $(a, b]$ (`Sturm.sturm`; los críticos son las raíces del producto de todos los miembros, un polinomio no nulo, luego finito). Quítese el mayor punto crítico c ; elíjase un separador no crítico d justo por debajo de él—existe porque los críticos son finitos y los reales son densos; aplíquese el cuanto por punto en $[d, b]$, donde c es el único punto crítico; y recúrrase en $[a, d]$, que tiene uno menos. El intervalo semiabierto $(a, b]$ se parte como $(a, d] \sqcup (d, b]$, el conteo de raíces se suma, y la contribución por punto—uno cuando $p(c) = 0$, cero en otro caso—coincide con el número de raíces de p en el trozo retirado. La escalera se ensambla escalón a escalón.

5. Q4: ¿qué queda gratis una vez certificado el teorema?

Apunte histórico. El conteo de Sturm de variaciones de signo en un *intervalo* y el conteo de Descartes de variaciones de signo en los *coeficientes* (1637) suelen enseñarse como primos. En Lean resultan ser, tras desplegar, la misma función—una pequeña sorpresa que la formalización sacó a la luz gratis. El Lean es `signVariations_eq_signChanges`.

Mathlib ya tenía la regla de los signos de Descartes, expresada por `Polynomial.signVariations`, el número de cambios de signo en la lista de coeficientes de un polinomio. El conteo construido aquí para el teorema de Sturm, `signChanges`, está definido sobre una lista arbitraria de signos. Al desplegar ambos, coinciden exactamente:

```
P.signVariations = signChanges(P.coeffList.map sign),
```

una prueba que es, literalmente, `rfl`. La consecuencia es práctica: el instrumental reunido para Sturm—los ceros son invisibles al conteo, una entrada entre vecinos opuestos se cancela, la recursión por cabeza, la paridad (una lista sin ceros tiene un número par de cambios de signo si y solo si sus extremos coinciden)—no es específico de Sturm. Se transfiere literalmente a Descartes, y los dos teoremas clásicos de variación de signo comparten ahora en la biblioteca un único núcleo combinatorio en vez de dos paralelos. Una pequeña unificación, pero exactamente del tipo que una biblioteca formal debería acumular.

Otras dos cosas caen sin trabajo nuevo. Primero, el *aislamiento de raíces*: aplicar el teorema a $(a, b]$ y a sus dos mitades repetidamente aísla cada raíz real en un intervalo tan pequeño como se quiera, el uso algorítmico por el que el método de Sturm es más conocido. Segundo, los *bloques de construcción* mismos—la constancia local del signo, el lema de vecinos antipodales, los hechos estructurales sobre la sucesión (miembros consecutivos coprimos, la recursión que relaciona cada terna, el último miembro una constante no nula)—son reutilizables para cualquier trabajo futuro de conteo de raíces reales en Lean: el teorema de Budan–Fourier, el conteo de Sturm–Tarski con multiplicidad [8], o un desarrollo del índice de Cauchy beberían de las mismas piezas.

6. Qué habilita un conteo de raíces certificado

La matemática aquí tiene dos siglos; la razón para ponerla en Lean es lo que desbloquea *dentro* de la biblioteca, donde un conteo exacto y verificado de raíces sencillamente no existía. Conviene ser concreto, porque un lema fundamental vale en proporción a lo que se apoya en él. Cuatro cosas se vuelven alcanzables que antes no lo eran, en orden aproximado de distancia a lo que se prueba.

Aislamiento de raíces reales. El uso algorítmico clásico del teorema de Sturm es aislar cada raíz real en un intervalo tan estrecho como se quiera: aplicar el conteo a $(a, b]$, biseccionar, recurrir, y detener una rama cuando su conteo es 0 (sin raíz) o 1 (exactamente una). Como el conteo aquí es *exacto*, no una cota, la recursión termina demostrablemente con una raíz por intervalo. Todos los ingredientes están ya probados—el conteo, la constancia local, la finitud del conjunto crítico—así que lo que queda es la envoltura recursiva y su terminación, no matemática nueva. Esto convierte el teorema de un enunciado en un *procedimiento* certificado, la forma en que el método de Sturm se usa de verdad.

El teorema de Budan–Fourier. Budan y Fourier cuentan variaciones de signo en la sucesión de *derivadas* $p, p', p'', \dots$ y acotan (con la paridad correcta) las raíces en un intervalo. Es la misma aritmética de variación de signo que aquí, sobre otra sucesión; el instrumental construido para Sturm—ceros invisibles, la recursión por cabeza, y en particular el lema de paridad `wallCount_even_iff_endpoints`—se transfiere directamente. Mathlib ya tenía la regla de Descartes (`signVariations`); Budan–Fourier se sitúa entre Descartes y Sturm, y las piezas para alcanzarlo están ahora en su sitio.

Sturm–Tarski y el índice de Cauchy. El teorema más fino de Sturm–Tarski cuenta raíces de p ponderadas por el signo de un segundo polinomio q sobre ellas—la “consulta de Tarski”—a través del índice de Cauchy de $q'p/q$. Este es el motor del conteo de raíces *con multiplicidad*, del caso no squarefree, y en última instancia de la eliminación de cuantificadores de Tarski para los reales. La sucesión de restos con signo y el conteo de variación ensamblados aquí son precisamente su sustrato; el desarrollo en Isabelle de Sturm–Tarski [7, 8] se construye sobre los mismos dos objetos. Una versión en Lean extendería, no reiniciaría, el presente fichero.

Procedimientos de decisión sobre los reales. El conteo y aislamiento de raíces reales en una variable son el caso base de la descomposición cilíndrica algebraica y de los procedimientos de decisión para la teoría de primer orden de los cuerpos realmente cerrados (Tarski–Seidenberg). Para Lean en concreto, este es el cimiento que faltaba bajo cualquier futura respuesta *certificada* a “¿tiene este polinomio una raíz en este rango, y cuántas?”—un certificado comprobable en el que una táctica aguas abajo podría confiar, en vez de un oráculo que deba creer. Ninguna de estas cuatro está hecha aquí; el punto es que cada una estaba bloqueada, en Lean, exactamente en el resultado que este artículo aporta.

7. Los bloques de construcción

El teorema descansa sobre un puñado de lemas reutilizables, cada uno libre de `sorry` con los tres axiomas estándar (Tabla 2). Tres merecen nombrarse por su portabilidad más allá de Sturm.

Lema 2 (constancia local del signo; `signVarAt_eq_of_no_root`). *Si ningún miembro de la sucesión tiene raíz en $[a, b]$, entonces $V(a) = V(b)$.*

El teorema del valor intermedio, elevado de un polinomio a la lista: esta es la planitud de la escalera entre puntos críticos, y el motor del caso base de la inducción global.

Lema 3 (la reducción por flancos preserva el conteo; `FlankReduce.signChanges_eq`). *Si una lista de signos se reduce por flancos a otra, ambas tienen el mismo número de cambios de signo.*

La mitad combinatoria de la Sección 4, que no depende de ningún polinomio—el enunciado más limpio de “las entradas entre vecinos opuestos no importan”.

Lema 4 (la estructura de la cadena; `sturmAux_consecutive_coprime`, `sturmAux_consecutive_succ`, `sturmAux_getLast_isUnit`). *Miembros consecutivos de la sucesión son coprimos; cada terna consecutiva cumple $p_{k+1} = -(p_{k-1} \bmod p_k)$; y el último miembro es una constante no nula.*

Estos tres hechos, probados por inducción sobre la recursión definitoria, son exactamente lo que consumen el Lema 1 y la inducción global.

nombre Lean	enunciado	papel
<code>Sturm.sturm</code>	$V(a) - V(b) = \#\{a < x \leq b : p(x) = 0\}$	Teorema 1
<code>sturmSeq</code>	la sucesión de restos con signo negado	definición
<code>signVarAt</code>	cambios de signo de la lista evaluada en x	definición
<code>signVarAt_drop_at_critical_point</code>	cuanto por punto $\Delta V \in \{0, 1\}$	Prop. 1
<code>flankReduce_chain_walk</code>	la cadena se reduce a sus no-anuladores	el muro (§4)
<code>FlankReduce.signChanges_eq</code>	la reducción preserva el conteo	núcleo combinatorio
<code>sign_neighbours_opposite_at_interior_root</code>	vecinos antipodales en un cero	Lema 1
<code>signVarAt_cons_head_drop</code>	un cambio de signo en la cabeza sube V en uno	cruce de raíz
<code>signVarAt_eq_of_no_root</code>	V constante fuera del conjunto crítico	constancia
<code>sturmAux_consecutive_coprime</code>	miembros consecutivos coprimos	estructura
<code>sturmAux_consecutive_succ</code>	$p_{k+1} = -(p_{k-1} \bmod p_k)$	estructura
<code>sturmAux_getLast_isUnit</code>	último miembro constante no nula	estructura
<code>signVariations_eq_signChanges</code>	$=$ conteo de Descartes (por <code>rfl</code>)	punto (§5)

Cuadro 2: Los resultados que sostienen todo, libres de `sorry` en `Sturm.lean`. Los axiomas de cada entrada son solo `propext`, `Classical.choice`, `Quot.sound`; sin `native_decide`, sin axioma propio.

Coste y reutilización

Como una formalización se juzga tanto por su coste como por su conclusión, aquí va el desarrollo medido (Tabla 3). Toda la prueba es un único fichero, `Sturm.lean`, de unas 1.220 líneas: 54 lemas y teoremas, 5 definiciones y una relación inductiva. Depende solo de `Mathlib`—doce módulos, listados en la fuente—y de nada de los artículos anteriores de esta línea; es autocontenido sobre la biblioteca. Sobre un `Mathlib` ya construido se comprueba en unos nueve segundos (Lean v4.30.0-rc2), de modo que el bucle de iteración durante el desarrollo fue ágil. Aproximadamente la mitad no trata de Sturm en absoluto: la teoría de variación de signo (`signChanges`, `wallCount`, `FlankReduce` y

los lemas de cirugía de listas) es una capa combinatoria independiente, reutilizable para cualquier argumento de conteo de signos —como muestra ya su identificación por `rfl` con el `signVariations` de Descartes (Sección 5). El resto, específico de Sturm, es la estructura de la sucesión y el análisis del cruce de lo local a lo global.

medida	valor
fuelle Lean	<code>Sturm.lean</code> , ≈ 1.220 líneas, un fichero
declaraciones	54 lemas/teoremas, 5 definiciones, 1 inductivo
dependencias	solo Mathlib (12 módulos); ninguna de trabajo previo
tiempo de compilación	≈ 9 s (solo el fichero, Mathlib precompilado), Lean v4.30.0-rc2
axiomas	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>
capa reutilizable	teoría de variación de signo, $\approx$ mitad del fichero, independiente de Sturm

Cuadro 3: El desarrollo medido. El tiempo de compilación es el de comprobar `Sturm.lean` contra un Mathlib ya construido; construir el propio Mathlib es el coste habitual de una sola vez y no forma parte de la contribución.

8. La frontera de la contribución

Permítaseme ser tan claro con los límites como con el resultado.

La matemática es de Sturm. No reclamo teorema nuevo ni método nuevo. La sucesión, el enunciado del conteo, el análisis local del cruce y la reducción a una diferencia de conteos de signo son todos clásicos (Sturm [3]; el tratamiento moderno en Basu–Pollack–Roy [4]). La contribución es la formalización, junto con la infraestructura reutilizable—el instrumental de variación de signo, el desacoplo `FlankReduce`, los hechos estructurales sobre la sucesión—que cualquier desarrollo futuro de raíces reales en Lean querría.

Alcance, dicho con llaneza. Pruebo el teorema para p *squarefree*. Es la normalización estándar y en principio no pierde generalidad: para p arbitrario se pasa a $p/\gcd(p, p')$, *squarefree* y con las mismas raíces distintas—pero *no* he formalizado esa reducción aquí, así que la hipótesis *squarefree* es una frontera genuina de lo verificado, no una mera comodidad. Trabajo sobre $\mathbb{R}$, el hogar natural del resultado y lo que usan los argumentos de valor intermedio y transversalidad, no sobre un cuerpo real cerrado abstracto. Se supone que los extremos no son raíces de p ; los miembros de la cola *pueden* anularse en los extremos, y ese caso se trata (Sección 4). Cuento raíces *distintas*; para p *squarefree* distintas y con multiplicidad coinciden, así que no hace falta un enunciado de multiplicidad aparte, pero tampoco se intenta el conteo de Sturm–Tarski con multiplicidades con signo [7, 8].

Dónde se sitúa, como contribución. Juzgada como matemática pura, la novedad es esencialmente nula—el teorema de Sturm es de 1829. Juzgada como formalización, creo que merece registrarse: un resultado clásico fundamental genuinamente ausente de una biblioteca importante, probado por completo con infraestructura reutilizable y una pieza de ingeniería no trivial. El lector natural es la comunidad de matemática formalizada y verificación más que la de matemática pura, y he intentado escribir para ella—de ahí la contabilidad de costes, el análisis de reutilización, y la comparación que sigue.

Formalizaciones previas, y qué difiere entre sistemas. La reivindicación de prioridad es estrecha y se apoya en una búsqueda documentada, no en una corazonada. El teorema de Sturm se ha formalizado antes: en Coq, dentro de la construcción de los números algebraicos reales de Cohen [5]; en Isabelle/HOL por Eberl [6] (un desarrollo autónomo con un método de prueba `sturm` ejecutable) y, en la forma más fina de Sturm–Tarski vía el índice de Cauchy, por Li y Paulson [7, 8]; y en

HOL Light. Lo que busqué (junio 2026) y no encontré fue ninguna formalización de la sucesión de Sturm ni del teorema de Sturm en *Lean*/Mathlib (Loogle y `grep: Polynomial.signVariations` para la regla de Descartes existe; no hay `sturm`, ni sucesión de restos con signo). Así: primero en Lean, hasta donde sé; no primero en ningún sistema.

Una palabra sobre lo que el sistema elegido costó y ahorró, que es lo que un lector de ingeniería de pruebas querrá. Lo que Mathlib *dio*, y volvió casi gratis varios pasos: la identidad de dominio euclídeo $p = (p \div q)q + (p \bmod q)$ con la cota de grado $\deg(p \bmod q) < \deg q$, que hace terminar la definición de la sucesión sin un argumento de medida aparte; `Polynomial.Splits`, `roots` y `mem_roots` para el conjunto de raíces; `separable` junto con `PerfectField.separable_iff_squarefree`, que entrega “squarefree $\Rightarrow$ coprimo con la derivada” en una línea; el teorema del valor intermedio (`intermediate_value_Icc`); y `Finset.max` con los lemas de cardinalidad para la inducción global. El sustrato analítico y algebraico, en otras palabras, ya estaba ahí. Lo que estaba *ausente en todo Lean* y hubo que construir es la teoría de variación de signo y todo el argumento de cruce de lo local a lo global; y el único paso genuinamente delicado—independiente del sistema—fue el acoplamiento de la recursión sobre listas que resuelve `FlankReduce` (Sección 4), no el análisis. No porté las pruebas de Cohen ni de Eberl, así que no afirmo nada sobre longitud o dificultad relativa: una comparación justa significaría rehacer el trabajo en cada sistema, lo que no he hecho. Lo que sí puedo decir es que los enfoques difieren en el marco—el de Cohen está embebido en aritmética real exacta, el de Eberl es ejecutable, el de Li–Paulson apunta al índice de Cauchy—y que, en Lean, el coste vivió en la combinatoria del conteo, no en el análisis polinómico que Mathlib ya sostiene.

Sobre el método. La formalización se hizo con una asistente de IA (Claude, Anthropic), bajo la disciplina usada en toda esta línea de trabajo: el ejemplo trabajado verificado en aritmética exacta antes de formalizar (el guion tras la Figura 2), y cada teorema con nombre comprobado por `#print axioms` para que descansen solo en `propext`, `Classical.choice`, `Quot.sound`. La corrección no depende de la asistente: el núcleo de Lean comprueba cada prueba, y un lector que confíe en el núcleo no necesita confiar en nada más. La contabilidad cuidadosa de esta sección—la frontera squarefree nombrada como frontera, los sistemas previos acreditados—es parte de lo que el método, usado con honestidad, debería producir.

9. Qué le falta todavía a este artículo

El teorema de Sturm está ya en la biblioteca; la pregunta más interesante es qué ocultaba su ausencia, y qué deja sin hacer este relato. Cierro con las preguntas que prefiero llevar adelante antes que fingir resueltas—cada una un lugar donde el trabajo presente se queda corto.

1. *La frontera squarefree.* Me apoyé en la ausencia de factores cuadrados y no formalicé el paso a $p/\gcd(p, p')$. Esa reducción es rutinaria sobre el papel; ¿lo es en Lean, o el máximo común divisor de un polinomio y su derivada esconde el tipo de contabilidad de grados que, como en la Sección 4, solo parece rutinario hasta que una máquina pregunta? Cerrarla haría el teorema incondicional, y es lo primero que haría a continuación.
2. *Los dos conteos, más allá de `rfl`.* Los conteos de variación de Descartes y de Sturm resultaron ser la misma función (Sección 5). Eso es una coincidencia de *definición*; ¿hay un teorema detrás—un único enunciado del que la cota de Descartes y el conteo exacto de Sturm sean dos instancias, y que una biblioteca pudiera guardar una vez en lugar de dos? El núcleo compartido es sugerente, pero solo he unificado la contabilidad, no los teoremas.
3. *Del conteo al certificado.* La prueba dice *cuántas* raíces hay en $(a, b]$; no devuelve, tal como está, las raíces mismas, ni un certificado comprobable del conteo para una herramienta escéptica

aguas abajo. Los bloques para el aislamiento de raíces están todos aquí (Sección 5); ¿cuál es el puente honesto más pequeño de este teorema a un procedimiento de aislamiento de raíces verificado—y cuánto de Sturm–Tarski [8] querría por el camino?

4. *Qué es de verdad el desacoplo.* La relación `FlankReduce` separó la combinatoria de podar una lista del álgebra de por qué cada poda es lícita (Sección 4). Funcionó con la limpieza suficiente como para sospechar que es una instancia de algo más general—un patrón para cualquier argumento donde un conteo es invariante bajo borrados autorizados por condiciones laterales locales. ¿Es “invariancia de un recuento bajo borrado localmente justificado” un lema reutilizable por derecho propio, o simplemente encajó aquí? Aún no lo sé, y esa incertidumbre es su estado honesto.

Reproducir

```
git clone https://github.com/karlesmarin/godsil-gutman-lean
cd godsil-gutman-lean && lake exe cache get
lake env lean Sturm.lean
```

Lean v4.30.0-rc2, Mathlib pin 701fb6e9. El teorema es `Sturm.sturm` en `Sturm.lean`; sus apoyos están en la Tabla 2. Se comprueba por `#print axioms` que depende solo de `propext`, `Classical.choice`, `Quot.sound`. El ejemplo trabajado de la Figura 2 se rederiva, y cada signo y conteo se afirma contra aritmética racional exacta, antes de dibujar la figura.

Referencias

- [1] R. Descartes, *La Géométrie* (1637); la regla de los signos acota el número de raíces positivas por el número de cambios de signo de los coeficientes.
- [2] F. D. Budan de Boislaurent, *Nouvelle méthode pour la résolution des équations numériques* (1807); J. Fourier, *Analyse des équations déterminées* (póstumo, 1831). El teorema de Budan–Fourier acota las raíces en un intervalo por una diferencia de variaciones de signo.
- [3] C. Sturm, *Mémoire sur la résolution des équations numériques*, *Mémoires présentés par divers savants à l’Académie Royale des Sciences* **6** (1835) 271–318; presentado a la Académie en 1829.
- [4] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, 2^a ed., Algorithms and Computation in Mathematics **10**, Springer, 2006 (teorema de Sturm y teoría de variación de signo, Cap. 2).
- [5] C. Cohen, Construction of real algebraic numbers in Coq, en *Interactive Theorem Proving (ITP 2012)*, LNCS **7406**, Springer, 2012, 67–82.
- [6] M. Eberl, Sturm’s Theorem, *Archive of Formal Proofs* (2014), https://isa-afp.org/entries/Sturm_Sequences.html.
- [7] W. Li, L. C. Paulson, A formal proof of the Sturm–Tarski theorem, *Archive of Formal Proofs* (2014), https://isa-afp.org/entries/Sturm_Tarski.html.
- [8] W. Li, The Budan–Fourier theorem and counting real roots with multiplicity, preprint, 2018, [arXiv:1811.11093](https://arxiv.org/abs/1811.11093); véase también W. Li, G. O. Passmore, L. C. Paulson, Deciding univariate polynomial problems using untrusted certificates in Isabelle/HOL, *J. Automated Reasoning* **62** (2019) 69–91.

- [9] The mathlib Community, The Lean mathematical library, en *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.
- [10] C. Marín, *Random Signs into Matchings: A Godsil–Gutman Identity, Formalized in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20517350.
- [11] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton’s Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20693064.